

TRABAJO PRÁCTICO INTEGRADOR

SISTEMA DE SEMÁFOROS INTELIGENTES

Asignatura: Paradigmas y Lenguajes de Programación.

Grupo 41

Integrantes:

- Sandoval, Diego Ulises
- Romero Fernandez, Ruben Tiburcio
- Sarli Ochat, Melina

1. INTRODUCCIÓN Y ENFOQUE DE DISEÑO

En este trabajo mostramos el diseño y desarrollo del núcleo lógico para el sistema de semáforos inteligentes. Para esto, implementamos la solución de las Fases 1 y 2 en Common Lisp y la Fase 3 en Scheme, cumpliendo con los principios del paradigma funcional.

Para garantizar el correcto funcionamiento de la intersección controlada, la solución se diseñó respetando los siguientes pilares:

- * **Inmutabilidad:** No se utilizaron variables globales mutables ni operadores destructivos como `setq` o `setf` para cambiar estados. El estado del tráfico fluye exclusivamente a través de los argumentos de las funciones y los retornos.
- * **Estructuras Funcionales:** Se evitaron los bucles imperativos tradicionales (`loop`, `while`). Toda la lógica de repetición o acumulación temporal se resolvió mediante composición funcional, recursividad y funciones de orden superior como `mapcar`.

2. FASE 2: AUTONOMÍA Y ECOSISTEMA (JUSTIFICACIÓN DE LIBRERÍA)

Para el desarrollo de la Fase 2, nuestro grupo seleccionó la integración de la librería externa `local-time` a través del gestor de paquetes `Quicklisp`.

Justificación Técnica:

Elegimos la librería `local-time` de `Quicklisp` porque para registrar los datos del tráfico (auditoría) es fundamental saber el momento exacto en que cambia una luz. Leer el tiempo en formato Unix (Epoch) sirve para que la computadora haga cálculos rápidos, pero para una persona que tiene que revisar los logs es muy difícil de entender. Con esta librería pasamos ese número a una fecha legible con formato de año, mes y día en texto plano.

3. BITÁCORA DE DEPURACIÓN (BUGS DETECTADOS Y RESUELTOS)

A continuación, se detallan los problemas sintácticos y lógicos que surgieron durante el desarrollo en Common Lisp y cómo los solucionamos para que coincidan con nuestro diseño final.

Bug 1: Error de Sintaxis por Paréntesis Anidados en transicion

- * **Síntoma:** El compilador arrojaba el error "Malformed COND clause" o intentaba ejecutar la lista de retorno como si fuera una función de evaluación lógica.
- * **Causa:** En la función de transición original colocamos la lista de retorno adentro de la condición lógica del `and`:

```
((and (eq color-actual 'en-verde) (eq cambiar-a 'rojo) (list 'en-verde "cambiar-a-rojo")))
```

Esto hacía que el intérprete evaluara `(list ...)` como una condición más del `and`, y al no encontrar qué acción ejecutar después, fallaba.

* **Solución:** Sacamos la expresión de retorno de los paréntesis del and, dejándola en el lugar correcto como la acción consecuente de esa rama del cond:

```
((and (eq color-actual 'en-verde) (eq cambiar-a 'rojo))
```

```
(list 'en-verde "cambiar-a-rojo"))
```

Bug 2: Manejo de efectos secundarios (Efectos Colaterales) en timer

* **Síntoma:** Al agregar la Extensión 2 (guardar el Log en un archivo de texto plano), necesitábamos que el sistema registre el cambio justo en el segundo exacto en que ocurre la transición de luces.

* **Causa:** Si dejábamos la función timer como un componente puramente matemático, necesitábamos otra función externa que controle los ciclos y llame al sistema de archivos, lo que complejizaba el flujo principal en esta etapa.

* **Solución:** Decidimos diseñar timer con una naturaleza impura de forma intencional. Ahora, cuando el residuo matemático de (mod hora-unix 225) detecta los segundos clave del cambio (0, 120, 123, 129, 132, 222), dispara directamente la función informe pasándole los estados y el timestamp, registrando el evento en el archivo "informeejecucion-semaforo.txt" sin romper la estructura de la función.

Bug 3: Uso de la hora de la computadora en lugar del tiempo simulado

* **Síntoma:** Al ejecutar pruebas en el simulador, todas las marcas de tiempo grabadas en el archivo de texto mostraban exactamente la misma hora (la hora real actual de la computadora).

* **Causa:** En el diseño inicial de la función formatear-hora estábamos usando la primitiva de sistema (local-time:now). Esto anulaba el sentido de la simulación, ya que el sistema debe procesar y registrar los eventos basados en el parámetro hora-unix que recibe de entrada, no en el reloj real actual de la máquina.

* **Solución:** Cambiamos la lógica para que use la variable hora-actual que entra como argumento. Usamos la función local-time:unix-to-timestamp para convertir ese valor recibido (unix) a un objeto timestamp tratable por la función local-time:formattimestring y asegurar que el reporte sea consistente con los datos simulados.

Bug 4: Error de Escala de Tiempo en la Distribución

* **Síntoma:** Al calcular los porcentajes de tiempo por hora en la función de distribución, los resultados daban valores menores al 10%, lo cual era incorrecto.

* **Causa:** En la variable hora-segundos pusimos por error 36000 en lugar de 3600. Ese cero de más multiplicaba por diez la base de segundos de una hora, provocando que los porcentajes calculados fueran diez veces más chicos.

* **Solución:** Corregimos el valor de la constante a 3600 dentro del let*, logrando que la función distribución-temporal calcule correctamente el porcentaje de ocupación de cada luz (incluyendo las intermitentes de 3 segundos) sobre el total de una hora.

4. FASE 3: ESTUDIO COMPARATIVO (SCHEME)

4.1 Presentación del Lenguaje Scheme

Scheme es un dialecto minimalista de Lisp diseñado en los años 70. A diferencia de Common Lisp, que es un estándar industrial más grande e integra múltiples paradigmas, Scheme se pensó bajo una filosofía de extrema simplicidad, donde pocas reglas bien combinadas resuelven problemas complejos.

* **Industrias y Aplicaciones:** Se usa mucho en investigación científica, educación informática y lenguajes de script integrados en sistemas complejos (como en el software GIMP, donde se usa bajo el nombre de Script-Fu).

* **Casos de Éxito Empresarial:** Cisco Systems lo utiliza para programar y automatizar componentes de infraestructura de redes. También es el motor principal en la herramienta de administración GNU Guix.

4.2 Respuestas Teóricas Obligatorias (Scheme)

1. Diferencia entre Lisp-1 y Lisp-2: La Ausencia de funcall y #' en Scheme

Un Lisp-2 (Common Lisp) maneja dos espacios de nombres (namespaces) independientes: uno para variables y otro para funciones. Si queremos pasar una función como parámetro, el compilador la busca por defecto en el espacio de variables. Por eso hay que usar el operador #' para avisarle que recupere la función y luego ejecutarla usando funcall o apply. Por otro lado, Scheme, al ser un Lisp-1, utiliza un único espacio de nombres para todos los identificadores. Las funciones se tratan como cualquier otra variable (objetos de primera clase). Por consiguiente, se pueden pasar por argumento o evaluar directamente en el primer lugar de una lista de ejecución sin necesidad de usar el símbolo #' ni recurrir a funcall.

2. Optimización de Llamada de Cola (TCO) y Prevención de Desbordamiento de Pila

Por estándar, los intérpretes de Scheme deben garantizar la Optimización de Llamada de Cola (TCO). Esta regla establece que si una función realiza una llamada recursiva en su posición de cola (es decir, la llamada recursiva es la última acción antes de retornar, sin dejar operaciones pendientes), el programa reutiliza el espacio en la pila de ejecución en vez de acumular memoria.

Afortunadamente, nuestras funciones de transición y timer no utilizan estrategias de recursividad, no obstante si así fuere la solución que proponemos sería la siguiente:

(define (bucle-semaforo tiempo-actual limite)

(cond

((>= tiempo-actual limite) (display "Simulación Finalizada"))

(else

(mostrar-estado (timer tiempo-actual))

:: LLAMADA DE COLA: No deja operaciones pendientes

(bucle-semaforo (+ tiempo-actual 1) limite))))

5. CONCLUSIONES

Este trabajo integrador nos permitió poner en práctica los conocimientos adquiridos durante la cursada, relacionados con la programación funcional en Common Lisp. Una de las principales dificultades fue adecuarnos a las limitaciones de la cátedra, sobre todo

en cuanto a la prohibición de uso de variables mutables y de las estructuras iterativas tradicionales. Esto hizo que tuviéramos que redefinir la manera en la que acostumbrábamos programar y buscar soluciones basadas en funciones puras, composición funcional y recursión, obteniendo una visión diferente sobre la resolución de problemas.

Desde el punto de vista organizativo, también fue una valiosa experiencia de trabajo en equipo. Aunque la coordinación del grupo no siempre fue como la habíamos planificado y dos miembros tuvieron que abandonar el proyecto en su desarrollo, esto nos obligó a reorganizar tareas, asumir nuevas responsabilidades y fortalecer la comunicación entre quienes seguimos trabajando. Estas situaciones nos dieron la oportunidad de aprender a gestionar proyectos colaborativos y a adaptarnos ante lo imprevisto.

En resumen, más allá de los resultados, este trabajo nos permitió desarrollar nuevas herramientas de pensamiento, conocer más profundamente el paradigma funcional y fortalecer habilidades de organización y colaboración que nos serán de utilidad en futuros proyectos académicos y profesionales.

6. BIBLIOGRAFÍA / FUENTES UTILIZADAS

- * Documentación oficial de Common Lisp (HyperSpec):
<https://www.lispworks.com/documentation/HyperSpec/>
- * Sitio oficial del lenguaje Scheme y especificaciones estándar:
<https://www.scheme.org/>

* Entorno de pruebas y ejecución interactiva Try Scheme: <https://try.scheme.org/> *

Guía de inicio de Quicklisp y gestión de librerías externas: <https://www.quicklisp.org/>

* Repositorio oficial y documentación de la librería local-time en GitHub:

<https://github.com/cl-pm/local-time>

* Repositorios comunitarios y ejemplos prácticos de manejo de archivos en Lisp (The Common Lisp Cookbook): <https://lispcookbook.github.io/cl-cookbook>